

Multi-agent systems - Homework 3

Matuš Halák (2724858), Thibault Giesbertz (2791192)

November 2025

1 Monte Carlo simulation and Kullback-Leibler divergence

1.1 KL divergence between normal distributions

Given two univariate gaussian distributions $f(x) = N(x|\mu, \sigma^2)$ and $g(x) = N(x|\nu, \tau^2)$, we look for a closed-form expression for the Kullback-Leibler (KL) divergence between two univariate gaussian distributions, with $KL(f||g)$ for any two distributions given by:

$$KL(f||g) = \int_{-\infty}^{\infty} f(x) \log\left(\frac{f(x)}{g(x)}\right) dx \equiv \mathbb{E}_f\left[\log\left(\frac{f(x)}{g(x)}\right)\right]$$

We express the full formula for a random variable x distributed according to the univariate gaussian $x \sim N(x|\mu, \sigma^2)$:

$$N(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

We know from probability theory that $\mathbb{E}[N(x|\mu, \sigma^2)] = \mu$ and that $\text{Var}(N(x|\mu, \sigma^2)) = \sigma^2$. Furthermore, using logarithm rules, we can express the log-likelihood of x :

$$\log N(x|\mu, \sigma^2) = -\frac{1}{2}(\log(2\pi) + \log(\sigma^2)) - \frac{1}{2\sigma^2}(x - \mu)^2$$

Similarly, using log-rules, we can rewrite the KL divergence formula:

$$KL(f||g) = \mathbb{E}_f[\log f(x) - \log g(x)]$$

then using linearity of expectation, we can express it as

$$KL(f||g) = \mathbb{E}_f[\log f(x)] - \mathbb{E}_f[\log g(x)]$$

Now we compute the individual terms, again using linearity of expectation

$$\mathbb{E}_f[\log f(x)] = \mathbb{E}_f\left[-\frac{1}{2}(\log(2\pi) + \log(\sigma^2)) - \frac{1}{2\sigma^2}(x - \mu)^2\right] = -\frac{1}{2}(\log(2\pi) + \log(\sigma^2)) - \frac{1}{2\sigma^2}\mathbb{E}_f[(x - \mu)^2]$$

In the last term, we recognize that $\mathbb{E}_f[(x-\mu)^2] = \mathbb{E}_f[(x-\mathbb{E}_f[x])^2] = \text{Var}_f[x] = \sigma^2$, which cancels out with the denominator. Therefore, the expression simplifies to

$$\mathbb{E}_f[\log f(x)] = -\frac{1}{2}(\log(2\pi) + \log(\sigma^2) + 1)$$

For the cross term, we follow the same logic, and now need to compute $\mathbb{E}_f[(x-v)^2]$. We expand the equation and apply expectation to the individual terms:

$$\begin{aligned}\mathbb{E}_f[(x-v)^2] &= \mathbb{E}_f[x^2] - 2\mathbb{E}_f[xv] + \mathbb{E}_f[v^2] \\ \mathbb{E}_f[x^2] &= \mathbb{E}_f[x^2] - \mathbb{E}_f[x]^2 + \mathbb{E}_f[x]^2 = \text{Var}_f[x] + \mathbb{E}_f[x]^2 = \sigma^2 + \mu^2 \\ \mathbb{E}_f[xv] &= v\mathbb{E}_f[x] = v\mu \\ \mathbb{E}_f[v^2] &= v^2 \\ \mathbb{E}_f[(x-v)^2] &= \sigma^2 + \mu^2 - 2v\mu + v^2 = \sigma^2 + (\mu - v)^2\end{aligned}$$

Then, plugging-in this result we obtain the final form of the second KL-divergence term

$$\mathbb{E}_f[\log g(x)] = -\frac{1}{2}(\log(2\pi) + \log(\tau^2) + \frac{\sigma^2 + (\mu - v)^2}{\tau^2})$$

Putting the two terms together we get:

$$\begin{aligned}\text{KL}(f||g) &= -\frac{1}{2}(\log(2\pi) + \log(\sigma^2) + 1) + \frac{1}{2}(\log(2\pi) + \log(\tau^2) + \frac{\sigma^2 + (\mu - v)^2}{\tau^2}) \\ &= \frac{1}{2}(2\log(\tau) - 2\log(\sigma) + \frac{\sigma^2 + (\mu - v)^2}{\tau^2} - 1) \\ \text{KL}(f||g) &= \frac{1}{2}(\log(\frac{\tau^2}{\sigma^2}) + \frac{\sigma^2 + (\mu - v)^2 - \tau^2}{\tau^2}) = \log(\frac{\tau}{\sigma}) + \frac{\sigma^2 + (\mu - v)^2 - \tau^2}{2\tau^2}\end{aligned}$$

Multiple stages of our proof make it immediately obvious that $\text{KL}(f||g)$ is not symmetric (i.e. $\text{KL}(f||g) \neq \text{KL}(g||f)$). Perhaps the most obvious comparison can be drawn at:

$$\text{KL}(f||g) = \mathbb{E}_f[\log f(x)] - \mathbb{E}_f[\log g(x)] \quad \text{KL}(g||f) = \mathbb{E}_g[\log g(x)] - \mathbb{E}_g[\log f(x)]$$

Where we see that the two equations contain completely different terms. From the derivations above, we can see what those terms would be and that they would lead to a different closed form solution, namely:

$$\text{KL}(g||f) = \log(\frac{\sigma}{\tau}) + \frac{\tau^2 + (v - \mu)^2 - \sigma^2}{2\sigma^2}$$

1.2 Monte Carlo estimate of KL divergence between normal distributions

```
import numpy as np
mu, sig2 = 3.1, 0.75
v, tau2 = -2.4, 2
N = int(1e7)
# Sample from  $f \sim N(\mu, \text{sig2})$ 
xf = np.random.normal(loc = mu, scale=np.sqrt(sig2), size = N)
# Sample from  $g \sim N(v, \text{tau2})$ 
xg = np.random.normal(loc = v, scale=np.sqrt(tau2), size = N)
# Univariate gaussian distribution
univar_gauss_pdf = lambda x, mean, var: np.exp(-((x - mean)**2)/(2*var))
                                     /np.sqrt(2*np.pi*var)

# Closed-form analytical solution
KLfg = 0.5*(np.log(tau2/sig2) + ((sig2-tau2+((mu - v)**2))/(tau2)))
KLgf = 0.5*(np.log(sig2/tau2) + ((tau2-sig2+((v - mu)**2))/(sig2)))
# MC solution  $KL(f||g)$ 
f = univar_gauss_pdf(xf, mu, sig2)
g = univar_gauss_pdf(xf, v, tau2)
MCKLfg = np.mean(np.log(f/g))
# MC solution  $KL(g||f)$ 
f = univar_gauss_pdf(xg, mu, sig2)
g = univar_gauss_pdf(xg, v, tau2)
MCKLgf = np.mean(np.log(g/f))
print(f'Analytical  $KL(f||g)$ :{KLfg}\n',
      f'Monte-Carlo  $KL(f||g)$ :{MCKLfg}\n',
      f'Analytical  $KL(g||f)$ :{KLgf}\n',
      f'Monte-Carlo  $KL(g||f)$ :{MCKLgf}')
'''

We obtain the output:
Analytical  $KL(f||g)$ :7.7404146265058635
Monte-Carlo  $KL(f||g)$ :7.741587131368327
Analytical  $KL(g||f)$ :20.509585373494136
Monte-Carlo  $KL(g||f)$ :20.510246849263076
'''
```

The code written above compares the theoretical closed-form solutions for the KL divergence between two univariate gaussian distributions, with a Monte-Carlo KL divergence estimate. We chose arbitrary distribution means and variances, leading to quite different distributions ($\mu = 3.1, \sigma^2 = 0.75$ and $v = -2.4, \tau^2 = 2$). We drew $1e-7$ random samples from $f(x)$ to estimate $KL(f||g)$, and $1e-7$ random samples from $g(x)$ to estimate $KL(g||f)$. The results (see commented print statement above) show that the Monte-Carlo estimates and analytical solutions for both $KL(f||g)$ and $KL(g||f)$ differ by less than $1e-2$. This confirms that our implementation is correct, and the results also empirically show that $KL(f||g) \neq KL(g||f)$ ($KL(f||g) = 7.74$, while $KL(g||f) = 20.51$)

2 Thompson Sampling for Bandits

2.1 Thompson update for single-arm bandit

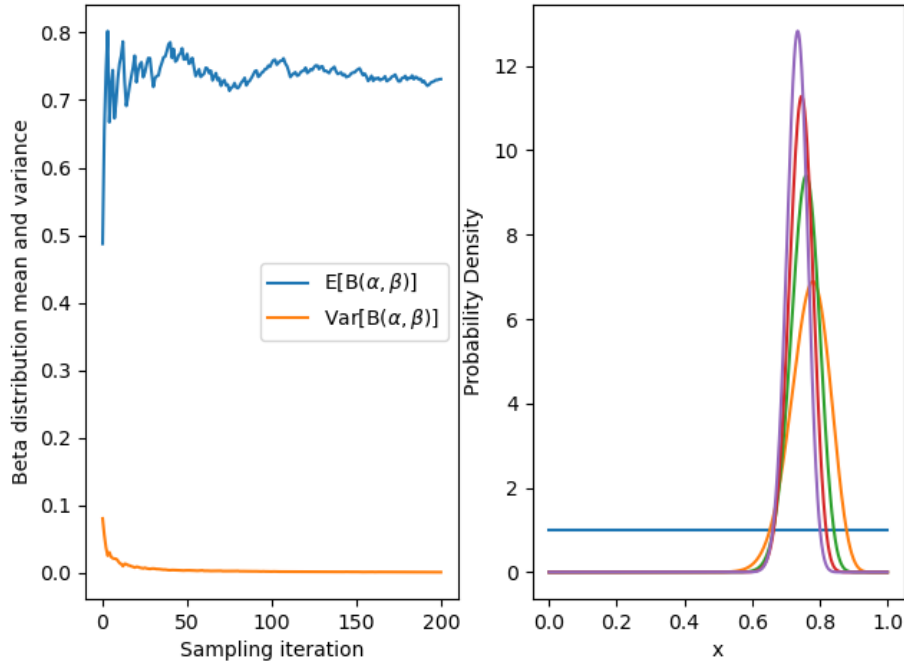


Figure 1: **Thompson sampling for single-arm bandit with ($p = 0.75$)**. Left: Expected value (mean), and variance of beta distribution across iterations. Right: Probability density sampled at 5 linearly spaced points between the first and last sampling iteration.

Figure 1 shows the results of our implementation of the Thompson update for a single-arm bandit (Code Appendix). We set the probability of reward to $p = 0.75$ and randomly sampled from the true binomial bandit distribution and used Thompson updates to update our beta distribution for 200 iterations. The variance of the beta distribution decreased rapidly over a few iterations. The expected value (mean) of the beta distribution converged to around $p = 0.73$ in the 200 iterations (we could get arbitrarily close to the true probability with more iterations, but those plots did not show the Thompson update process as nicely, since the distributions become very peaked and are only minorly adjusted after approximately 200 iterations). Our plot also shows that initially, due to random sampling, the probability of a reward was slightly over-estimated, and that over time, the Thompson update centers the Beta probability distribution around the true probability of reward.

2.2 Thompson and UCB sampling for a general K-arm bandit (CODE)

```
def bandit(n_iter:int, probs:list[float], mode:Literal['thompson', 'ucb'],
           c_param:float = None):
    K = len(probs) # number of bandits

    if mode == 'ucb':
        assert c_param is not None
        # Q-values for different arms
        qvals = np.zeros(K)
        # N times each arm pulled
        pulls = np.zeros(K)
        # for comparison with thompson 0:failure, 1:success
        fail_succes = np.zeros((K, 2))

        for t in range(n_iter):
            # Start by pulling each arm at least once
            if np.any(pulls < 1):
                chosen_arm = np.argmin(pulls)
            else: # then follow UCB rule
                ucb = c_param * np.sqrt(np.log(t) / pulls)
                ucb += qvals / pulls
                chosen_arm = np.argmax(ucb)
            reward = rand.binomial(n=1, p = probs[chosen_arm])
            qvals[chosen_arm] += reward
            pulls[chosen_arm] += 1
            # for comparison with thompson
            fail_succes[chosen_arm, reward] += 1

        # Est. Posterior probabilities of success per arm (#success / (#pulls))
        print(fail_succes[:, 1] / fail_succes.sum(axis=1))

    elif mode == 'thompson':
        # col1 = a, col2 = b (initialized to 1,1)
        beta_params = np.ones((K, 2))
        # 1: success (a), 0: failure (b)
        beta_update = {1:0, 0:1}

        for iter in range(n_iter):
            simulated_outcomes = rand.beta(a = beta_params[:, 0], b = beta_params[:, 1])
            best_arm = np.argmax(simulated_outcomes)
            reward = rand.binomial(n=1, p = probs[best_arm])
            beta_params[best_arm, beta_update[reward]] += 1

        # Est. Posterior probabilities of success per arm (#success / (#pulls))
        print(beta_params[:, 0] / beta_params.sum(axis=1))
```

Setting the probabilities for three bandits to $p_1 = 0.1, p_2 = 0.6, p_3 = 0.85$ and doing 1000 total pulls using thompson sampling,

```
bandit(n_iter=1000, probs=[0.1, 0.6, 0.85], mode='thompson')
```

we obtain the following posterior probabilities of success for each arm: $\hat{p}_1 = 0.1666, \hat{p}_2 = 0.5555, \hat{p}_3 = 0.8564$, which is already a good and informative estimate.

2.3 Thompson sampling versus UCB (Experiments)

We define total regret over a set of actions $\{a_t\}_1^T$ taken over a finite horizon T , as the sum of regrets of individual actions $R(a_t)$, which measures the difference between the maximum expected reward obtainable with the optimal action p^* and the expected reward of the chosen action $p(a_t)$.

$$R(\{a_t\}_1^T) = \sum_t^T R(a_t) = \sum_t^T p^* - p(a_t)$$

Because we are working with bandits that all produce a binary reward, the expected reward of the i th bandit is simply the probability that it produces the reward of 1 at any given pull $p(r = 1|k = i)$.

Because we are in control of the bandit setup, we have ground-truth information of what p^* is, and can simply keep track of the cumulative regret as sampling progresses for a fixed number of iterations, by adding the following lines of code.

```
...
# expected reward of best arm = maximum success probability across bandits
pstar = max(probs)
# regrets
total_regret = 0
cum_regret = np.zeros(n_iter)
...
for t in range(n_iter):
    ...
    # update total regret
    total_regret += pstar - probs[chosen_arm]
    cum_regret[t] = total_regret
    ...
```

We investigated the hyperparameter c across three orders of magnitude over four settings $c \in (0.25, 1, 2, 20)$. We used the same three-armed bandit problem from the previous question ($p_1 = 0.1, p_2 = 0.6, p_3 = 0.85$) and sampled 1000 times using each UCB method, as well as, using Thompson sampling.

Figure 2 shows the results of our numerical experiments. We see that increasing the c hyperparameter for UCB sampling increases exploration of the

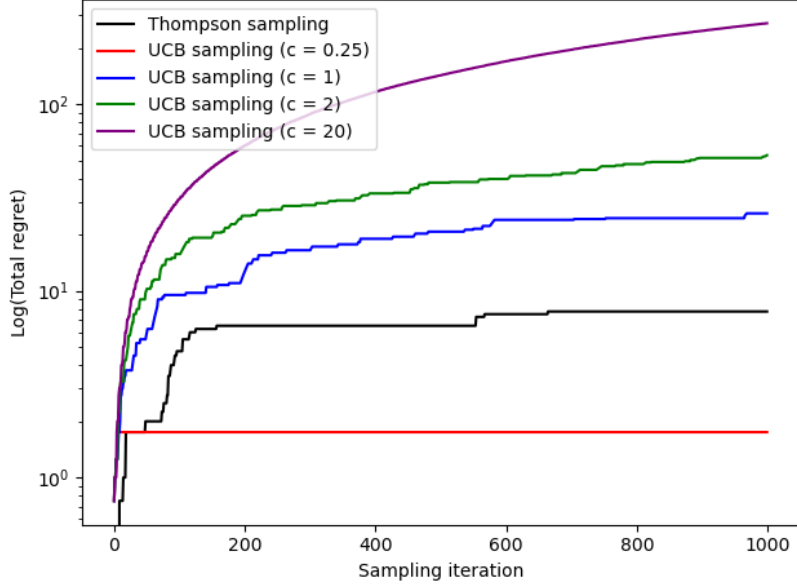


Figure 2: (Log) Total regret with Thompson vs UCB sampling.

unfavorable arms, leading to higher total regret. Setting the hyperparameter c to a very low value (0.25) discourages exploration except in the initial K iterations where all arms must be tried at least once. Afterwards, a UCB rule with a low c -value always exploits the arm with the highest Q -value. A disadvantage of this approach is that because the rewards are stochastic, in the unlikely case the best arm does not produce a reward in the initial pulls, this decision rule would keep exploiting the wrong arm for unnecessarily long. This would be especially problematic if the reward probabilities were lower and closer to one another between the individual arms.

We notice that Thompson sampling strikes a better balance between exploration and exploitation, while not requiring any hyperparameters. Namely, disregarding the highly exploitative setting with low c , which will fail in scenarios outlined above, Thompson sampling led to a lower total regret than UCB sampling with $c \in (1, 2, 20)$. From the total regret curve, we notice that Thompson sampling initially performs exploration of each arm (marked by steep increase in total regret over initial sampling iterations), after which it stabilizes and keeps picking the best arm, effectively showing a constant total regret for most of the sampling iterations.

A Code Appendix

A.1 Thompson update for single-arm bandit

```
import numpy as np
import numpy.random as rand
from math import factorial as fact
from matplotlib import pyplot as plt

# Experiment setup
n_trials = int(2e2)+1
npdfs = 4
pdfplot = n_trials // npdfs
n_samples = int(1e3)
p_bandit = 0.75 # target to estimate

# Data collection
mean_s = np.zeros(n_trials)
var_s = np.zeros(n_trials)
beta_pdfs = np.zeros((npdfs+1, n_samples))

# Beta distribution essentials
beta_prior = dict(a = 1, b = 1) # uniform prior
beta_pdf = lambda x, a, b: (fact(a+b-1)/(fact(a-1)*fact(b-1))
                             ) * (x**(a-1)) * ((1-x)**(b-1))
update_beta = {1: 'a', 0: 'b'}

for t in range(n_trials):
    if t % pdfplot == 0:
        # Beta distribution density measured regularly
        beta_pdfs[t//pdfplot, :] = beta_pdf(np.linspace(0,1, n_samples), **beta_prior)
        beta_dist = rand.beta(**beta_prior, size = n_samples)
        mean_s[t] = np.mean(beta_dist) # empirically measure mean
        var_s[t] = np.var(beta_dist) # empirically measure variance
        # pull bandit arm
        reward = rand.binomial(n = 1, p = p_bandit, size = 1)[0]
        beta_prior[update_beta[reward]] += 1 # thompson update

print(pdfplot)
f, ax = plt.subplots(ncols=2)
ax[0].plot(mean_s, label = r'E[B($\alpha, \beta$)]')
ax[0].plot(var_s, label = r'Var[B($\alpha, \beta$)]')
ax[0].legend(); ax[0].set_xlabel('Sampling iteration')
ax[0].set_ylabel('Beta distribution mean and variance')
ax[1].plot(np.linspace(0,1, n_samples), beta_pdfs.T)
ax[1].set_ylabel('Probability Density'); ax[1].set_xlabel('x')
plt.show()
```